

■ Security Analysis Report

Generated: 2026-02-13 14:54:10

Summary

Metric	Value
Total Vulnerabilities	33
Security Score	0/100
Critical	9
High	13
Medium	11
Low	0

Vulnerabilities

[1] Weak Session Management

Severity	High
Category	A07
File	test_samples\secure.php
Line	5
Rule ID	A07-PHP-001

Description: Missing session security configuration

Code:

```
session_start(); $users = [
```

Remediation: Configure secure session settings: httpOnly, secure flag, SameSite attribute, regenerate session ID on login

[2] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\secure.php
Line	13
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
$username = filter_input(INPUT_POST, "username", FILTER_SANITIZE_SPECIAL_CHARS);  
$password = $_POST["password"] ?? ""; if (!$username || !$password) {
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[3] Taint Analysis: SQL Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\taint_demo.js
Line	10
Rule ID	TAINT-SQL-INJECTION

Description: Tainted data from URL query parameters (line 9) flows through variable 'query' to Database query

Code:

```
const query = "SELECT * FROM users WHERE id = " + userId; db.query(query); // SINK - SQL Injection! };
```

Remediation: Sanitize data from 'URL query parameters' before using in 'Database query'. Data flow:
Source: URL query parameters -> Propagated to query

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\taint_demo.js

[4] Taint Analysis: Command Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\taint_demo.js
Line	24
Rule ID	TAINT-COMMAND-INJECTION

Description: Tainted data from POST request body (line 23) flows through variable 'cmd' to Command execution

Code:

```
const cmd = "tar -czf backup.tar.gz " + filename; exec(cmd); // SINK - Command Injection! };
```

Remediation: Sanitize data from 'POST request body' before using in 'Command execution'. Data flow: Source: POST request body -> Propagated to cmd

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\taint_demo.js

[5] Taint Analysis: Code Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\taint_demo.js
Line	36
Rule ID	TAINT-CODE-INJECTION

Description: Tainted data from POST request body (line 35) flows through variable 'expression' to Code evaluation

Code:

```
const expression = req.body.expr; // TAINT SOURCE const result = eval(expression); //  
SINK - Code Injection! res.json({ result }); });
```

Remediation: Sanitize data from 'POST request body' before using in 'Code evaluation'. Data flow:
Source: POST request body

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAINTED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\taint_demo.js

[6] Taint Analysis: SQL Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\taint_demo.js
Line	45
Rule ID	TAINT-SQL-INJECTION

Description: Tainted data from URL query parameters (line 9) flows through variable 'query' to Database query

Code:

```
const query = "SELECT * FROM users WHERE id = " + safeId; db.query(query); // Should be safe });
```

Remediation: Sanitize data from 'URL query parameters' before using in 'Database query'. Data flow:
Source: URL query parameters -> Propagated to query

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\taint_demo.js

[7] Unsafe Deserialization

Severity	High
Category	A08
File	test_samples\taint_demo.js
Line	36
Rule ID	A08-JS-001

Description: Unsafe deserialization of untrusted data

Code:

```
const expression = req.body.expr; // TAIN'T SOURCE const result = eval(expression); //  
SINK - Code Injection! res.json({ result }); });
```

Remediation: Validate and sanitize data before deserialization, use safe parsing methods

[8] Unsafe Deserialization

Severity	High
Category	A08
File	test_samples\vuljs1.js
Line	24
Rule ID	A08-JS-001

Description: Unsafe deserialization of untrusted data

Code:

```
// ■ Dangerous use of eval (Code Injection) app.get("/calc", (req, res) => { const  
result = eval(req.query.exp);
```

Remediation: Validate and sanitize data before deserialization, use safe parsing methods

[9] Unsafe Deserialization

Severity	High
Category	A08
File	test_samples\vuljs1.js
Line	26
Rule ID	A08-JS-001

Description: Unsafe deserialization of untrusted data

Code:

```
app.get("/calc", (req, res) => { const result = eval(req.query.exp); res.send("Result: " + result); });
```

Remediation: Validate and sanitize data before deserialization, use safe parsing methods

[10] Weak Password Hashing

Severity	Critical
Category	A02
File	test_samples\vulnerable.js
Line	17
Rule ID	A02-JS-001

Description: Use of weak cryptographic hash functions (MD5, SHA1)

Code:

```
function hashPassword(password) { return  
  crypto.createHash("md5").update(password).digest("hex"); }
```

Remediation: Use strong hashing algorithms like bcrypt, argon2, or PBKDF2 with sufficient iterations

[11] Hardcoded Secrets

Severity	High
Category	A05
File	test_samples\vulnerable.js
Line	22
Rule ID	A05-JS-001

Description: Hardcoded credentials or API keys in source code

Code:

```
const config = { api_key: "sk_live_1234567890abcdef", password: "admin123", };
```

Remediation: Store secrets in environment variables or secure secret management systems

[12] Hardcoded Secrets

Severity	High
Category	A05
File	test_samples\vulnerable.js
Line	23
Rule ID	A05-JS-001

Description: Hardcoded credentials or API keys in source code

Code:

```
api_key: "sk_live_1234567890abcdef", password: "admin123", };
```

Remediation: Store secrets in environment variables or secure secret management systems

[13] Taint Analysis: SQL Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	7
Rule ID	TAINT-SQL-INJECTION

Description: Tainted data from User input from URL parameters (line 6) flows through variable 'query' to MySQL query execution

Code:

```
$query = "SELECT * FROM users WHERE id = " . $userId; $result = mysqli_query($conn, $query); // A03: XSS
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'MySQL query execution'. Data flow: Source: User input from URL parameters -> Propagated to query

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAINTED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\vulnerable.php

[14] Taint Analysis: Command Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	23
Rule ID	TAINT-COMMAND-INJECTION

Description: Tainted data from User input from URL parameters (line 22) flows through variable 'filename' to Shell command execution

Code:

```
$filename = $_GET['file']; exec("cat " . $filename); // A10: SSRF
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'Shell command execution'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\vulnerable.php

[15] Taint Analysis: Path Traversal

Severity	High
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	27
Rule ID	TAINT-PATH-TRAVERSAL

Description: Tainted data from User input from URL parameters (line 26) flows through variable 'url' to File system operation

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'File system operation'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\vulnerable.php

[16] Taint Analysis: Path Traversal

Severity	High
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	27
Rule ID	TAINT-PATH-TRAVERSAL

Description: Tainted data from User input from URL parameters (line 26) flows through variable 'url' to Remote file access

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'Remote file access'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\vulnerable.php

[17] Taint Analysis: Server-Side Request Forgery

Severity	High
Category	Data Flow Vulnerability
File	test_samples\vulnerable.php
Line	27
Rule ID	TAINT-SERVER-SIDE-REQUEST-FORGERY

Description: Tainted data from User input from URL parameters (line 26) flows through variable 'url' to Remote file access

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Sanitize data from 'User input from URL parameters' before using in 'Remote file access'. Data flow: Source: User input from URL parameters

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAIN'TED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\vulnerable.php

[18] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	5
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A03: SQL Injection $userId = $_GET['id']; $query = "SELECT * FROM users WHERE id = "
. $userId; $result = mysqli_query($conn, $query);
```

Remediation: Validate and sanitize all input using filter_input() or filter_var()

[19] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	10
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A03: XSS echo $_GET['name']; print $_POST['comment'];
```

Remediation: Validate and sanitize all input using filter_input() or filter_var()

[20] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	11
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
echo $_GET['name']; print $_POST['comment']; // A02: Weak Password Hashing
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[21] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	14
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A02: Weak Password Hashing $password = $_POST['password']; $hash = md5($password);
```

Remediation: Validate and sanitize all input using filter_input() or filter_var()

[22] Weak Password Hashing

Severity	Critical
Category	A02
File	test_samples\vulnerable.php
Line	15
Rule ID	A02-PHP-001

Description: Use of weak cryptographic hash functions

Code:

```
$password = $_POST['password']; $hash = md5($password); // A05: Hardcoded Secrets
```

Remediation: Use password_hash() with PASSWORD_BCRYPT or PASSWORD_ARGON2ID

[23] Hardcoded Secrets

Severity	High
Category	A05
File	test_samples\vulnerable.php
Line	18
Rule ID	A05-PHP-001

Description: Hardcoded credentials or secrets in source code

Code:

```
// A05: Hardcoded Secrets $password = "admin123"; $api_key =  
"sk_live_1234567890abcdef";
```

Remediation: Use environment variables or secure configuration files outside web root

[24] Hardcoded Secrets

Severity	High
Category	A05
File	test_samples\vulnerable.php
Line	19
Rule ID	A05-PHP-001

Description: Hardcoded credentials or secrets in source code

Code:

```
$password = "admin123"; $api_key = "sk_live_1234567890abcdef"; // A03: Command  
Injection
```

Remediation: Use environment variables or secure configuration files outside web root

[25] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	22
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A03: Command Injection $filename = $_GET['file']; exec("cat " . $filename);
```

Remediation: Validate and sanitize all input using filter_input() or filter_var()

[26] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulnerable.php
Line	26
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// A10: SSRF $url = $_GET['url']; $content = file_get_contents($url);
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[27] Server-Side Request Forgery

Severity	High
Category	A10
File	test_samples\vulnerable.php
Line	27
Rule ID	A10-PHP-001

Description: File/URL access with user-controlled input, vulnerable to SSRF

Code:

```
$url = $_GET['url']; $content = file_get_contents($url); // A07: Weak Session Management
```

Remediation: Validate URLs, whitelist allowed domains, block internal network access

[28] Weak Session Management

Severity	High
Category	A07
File	test_samples\vulnerable.php
Line	30
Rule ID	A07-PHP-001

Description: Missing session security configuration

Code:

```
// A07: Weak Session Management session_start(); // Missing session security  
configuration
```

Remediation: Configure secure session settings: httpOnly, secure flag, SameSite attribute, regenerate session ID on login

[29] Taint Analysis: SQL Injection

Severity	Critical
Category	Data Flow Vulnerability
File	test_samples\vulphp.php
Line	12
Rule ID	TAINT-SQL-INJECTION

Description: Tainted data from User input from POST data (line 11) flows through variable 'query' to MySQL query execution

Code:

```
$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";  
$result = mysqli_query($conn, $query); // ■ Authentication bypass possible
```

Remediation: Sanitize data from 'User input from POST data' before using in 'MySQL query execution'. Data flow: Source: User input from POST data -> Propagated to query

TAINT DATA FLOW ANALYSIS



■ SECURITY VULNERABILITY: TAINED DATA REACHES SINK WITHOUT SANITIZATION ■

File: test_samples\vulphp.php

[30] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulphp.php
Line	7
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
$username = $_POST["username"]; $password = $_POST["password"];
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[31] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulphp.php
Line	8
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
$username = $_POST["username"]; $password = $_POST["password"]; // ■ SQL Injection
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[32] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulphp.php
Line	22
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
// ■ Command Injection if (isset($_GET["cmd"])) { system($_GET["cmd"]); }
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`

[33] Missing Input Validation

Severity	Medium
Category	Best Practice
File	test_samples\vulphp.php
Line	23
Rule ID	BP-PHP-001

Description: Direct use of superglobals without validation

Code:

```
if (isset($_GET["cmd"])) { system($_GET["cmd"]); } ?>
```

Remediation: Validate and sanitize all input using `filter_input()` or `filter_var()`